Edeviser — Edge Functions Deployment Plan

Inventory, Deployment Strategy, Supabase Limitations & Improvement Suggestions

Date: April 2026 | **Project:** Edeviser-Kiro (cdlgtbvxlxjpcddjazzx) | **Region:** ap-northeast-1

---

1. Current State

36 Edge Functions exist as local code in `supabase/functions/`. Only 1 (health) is deployed to Supabase production. 35 functions await deployment.

Metric	Value
Total local functions	36
Deployed to Supabase	1 (health)
Pending deployment	35
Shared utilities	1 (_shared/rateLimiter.ts)
Supabase plan	Pro
Database version	PostgreSQL 17.6

---

2. Complete Edge Function Inventory

Batch 1: Core Platform (Deploy at Pilot Launch)

These are required for the platform to function. No external API keys needed.

Function	Purpose	JWT Required	Secrets Needed
award-xp	XP engine: insert transaction, recalculate total, check level-up, apply bonus multiplier	Yes	None
process-streak	Streak engine: increment/reset streak on login, check milestones	Yes	None
check-badges	Badge engine: idempotent badge condition check, insert awards	Yes	None
calculate-attainment-rollup	Evidence generator: CLO to PLO to ILO weighted rollup, UPSERT attainment	Yes	None
check-login-rate	Login rate limiter: 5 attempts per 15 min per IP	No	None
auto-grade-quiz	Auto-grade MCQ/T-F quizzes, calculate scores	Yes	None
bulk-import-users	CSV bulk user creation with email invitations	Yes	RESEND_API_KEY
bulk-data-import	Generic CSV data import (courses, enrollments)	Yes	None
export-student-data	GDPR data export for student records	Yes	None
improvement-bonus-check	Check and award improvement bonuses	Yes	None

Batch 2: Email & Notifications (Deploy After Resend Key Configured)

Function	Purpose	JWT Required	Secrets Needed
send-email-notification	Transactional email via Resend (streak risk, weekly summary, grade released)	Yes	RESEND_API_KEY
notification-digest	Batch and send notification digests	Yes	RESEND_API_KEY
perfect-day-prompt	6 PM cron: check 3/4 habits, send nudge notification	No (cron)	None

Batch 3: AI Co-Pilot (Deploy After OpenRouter Key Configured)

Function	Purpose	JWT Required	Secrets Needed
ai-module-suggestion	AI-powered learning module suggestions based on CLO gaps	Yes	OPENROUTER_API_KEY
ai-at-risk-prediction	AI at-risk student detection from behavioral signals	Yes	OPENROUTER_API_KEY
ai-feedback-draft	AI-generated rubric feedback drafts for teachers	Yes	OPENROUTER_API_KEY
compute-at-risk-signals	Nightly cron: compute behavioral risk signals from activity log	No (cron)	None

Batch 4: Adaptive Quiz (Deploy After OpenRouter Key Configured)

Function	Purpose	JWT Required	Secrets Needed
generate-quiz-questions	AI question generation from course materials via RAG	Yes	OPENROUTER_API_KEY
select-adaptive-question	Per-student adaptive question selection based on ability	Yes	None
update-question-analytics	Post-quiz analytics: success rate, discrimination index, calibrated difficulty	Yes	None

Batch 5: Reports & Documents (Deploy at Pilot Launch)

Function	Purpose	JWT Required	Secrets Needed
generate-accreditation-report	PDF accreditation report generation (jspdf)	Yes	None
generate-course-file	Course file generation for accreditation	Yes	None
generate-transcript	Student transcript PDF generation	Yes	None
generate-fee-receipt	Fee payment receipt generation	Yes	None

Batch 6: Student Onboarding (Deploy at Pilot Launch)

Function	Purpose	JWT Required	Secrets Needed
process-onboarding	Score Big Five, VARK, self-efficacy assessments	Yes	None
suggest-goals	AI-generated SMART goal suggestions	Yes	OPENROUTER_API_KEY
generate-starter-week	AI-generated first week study plan	Yes	OPENROUTER_API_KEY

Batch 7: Cron Jobs (Deploy After pg_cron Configured)

These functions are invoked by pg_cron, not by client requests.

Function	Purpose	Schedule	Secrets Needed
streak-risk-cron	Daily 8 PM: email students at risk of losing streak	Daily	RESEND_API_KEY
fee-overdue-check	Daily 6 AM: update pending fees to overdue	Daily	None
exam-period-notify	Notify students of upcoming exam periods	As configured	RESEND_API_KEY
team-streak-risk-cron	Daily: check team streak health	Daily	None
compute-habit-correlations	Weekly: compute habit-academic correlations	Weekly	None

Batch 8: Team Challenges (Deploy When Feature Enabled)

Function	Purpose	JWT Required	Secrets Needed
challenge-completion	Process social challenge completion, award team XP	Yes	None
challenge-progress-update	Update challenge progress on student actions	Yes	None

Batch 9: Data Import (Deploy When Needed)

Function	Purpose	JWT Required	Secrets Needed
import-competency-csv	Import competency framework data from CSV	Yes	None

Already Deployed

Function	Status
health	Active (deployed, JWT disabled)

---

3. Required Secrets Configuration

Before deploying functions that call external APIs, these secrets must be set in the Supabase Dashboard (Project Settings > Edge Functions > Secrets):

Secret	Required By	How to Obtain
OPENROUTER_API_KEY	ai-module-suggestion, ai-at-risk-prediction, ai-feedback-draft, generate-quiz-questions, suggest-goals, generate-starter-week	Sign up at openrouter.ai, create API key
RESEND_API_KEY	send-email-notification, notification-digest, streak-risk-cron, exam-period-notify, bulk-import-users	Sign up at resend.com, verify domain, create API key
SUPABASE_SERVICE_ROLE_KEY	All functions using service role client	Already available in Supabase Dashboard
SUPABASE_URL	All functions	Already available in Supabase Dashboard

---

4. Deployment Priority & Timeline

Priority	Batch	Functions	When	Prerequisite
P0	Batch 1 (Core)	10 functions	Pilot launch	None
P0	Batch 5 (Reports)	4 functions	Pilot launch	None
P0	Batch 6 (Onboarding)	3 functions (process-onboarding only; goals/starter-week need AI key)	Pilot launch	OPENROUTER_API_KEY for 2 of 3
P1	Batch 2 (Email)	3 functions	After RESEND_API_KEY configured	RESEND_API_KEY
P1	Batch 3 (AI Co-Pilot)	4 functions	After OPENROUTER_API_KEY configured	OPENROUTER_API_KEY
P1	Batch 4 (Adaptive Quiz)	3 functions	After OPENROUTER_API_KEY configured	OPENROUTER_API_KEY
P2	Batch 7 (Cron Jobs)	5 functions	After pg_cron enabled on Pro plan	pg_cron extension
P2	Batch 8 (Team Challenges)	2 functions	When team features activated	None
P3	Batch 9 (Data Import)	1 function	When needed	None

---

5. Supabase Edge Function Limitations

5.1 Plan Limits (Pro Plan)

Limit	Pro Plan Value	Impact on Edeviser
Invocations included	2,000,000/month	Sufficient for pilot (500 users). At 10K users, may need monitoring.
Overage cost	$2 per 1M invocations	Predictable scaling cost
Max execution time	150 seconds (wall clock)	AI functions (quiz generation, feedback drafts) may approach this limit for large batches
Max request body	2MB	Sufficient for most operations. Bulk CSV imports may need chunking for large files.
Max response body	No hard limit, but 6MB recommended	PDF generation (accreditation reports) may produce large responses. Use Supabase Storage for large files.
Concurrent connections	500 (Pro) / 1,000 (Team)	During exam periods with 500+ students submitting simultaneously, connection limits may be hit
Cold start time	200-500ms typical	First invocation after idle period is slower. Affects user-perceived latency for infrequent functions.
Runtime	Deno (not Node.js)	Cannot use npm packages directly. Must use esm.sh or Deno-compatible imports. Limits library ecosystem.
Secrets	50 max per project	Sufficient for current needs (4 secrets)
Regions	Single region per project	Functions run in ap-northeast-1 (Tokyo). Qatar users experience 100-200ms additional latency vs. a Middle East region.

5.2 Known Constraints

1. No WebSocket support in Edge Functions. Realtime features must use Supabase Realtime (Phoenix Channels), not Edge Functions.
2. No persistent state between invocations. Each function invocation is stateless. Use the database for any state that needs to persist.
3. No file system access. Functions cannot read/write files. Use Supabase Storage for file operations.
4. No background processing. Functions must complete within the execution time limit. Long-running tasks (large PDF generation, bulk AI processing) need to be chunked or use a queue pattern.
5. Deno runtime only. The npm ecosystem is not directly available. Libraries must be imported via esm.sh or jsr. Some Node.js libraries have no Deno equivalent.
6. No cron scheduling built into Edge Functions. Cron jobs require pg_cron extension (Pro plan) to trigger functions via pg_net HTTP calls.
7. Single region deployment. Functions deploy to the project's region only. No multi-region or edge deployment for lower latency.

---

6. Improvement Suggestions

6.1 Before Deployment

1. Set up a staging branch. Use Supabase branching to test Edge Function deployments before production. This prevents broken functions from affecting the live database.
2. Configure all secrets first. Set OPENROUTER_API_KEY and RESEND_API_KEY in the Supabase Dashboard before deploying functions that depend on them. Functions will fail silently if secrets are missing.
3. Add health checks to each function. Every function should return a 200 OK on GET requests for monitoring. The existing health function pattern should be replicated.
4. Implement request validation. Every function should validate the request body with Zod before processing. This prevents malformed requests from causing errors.

6.2 Performance Optimization

5. Add response caching for AI functions. AI module suggestions and at-risk predictions for the same student/CLO combination can be cached for 24 hours. This reduces OpenRouter API costs by 40-60%.
6. Batch quiz question generation. Instead of generating questions one at a time, batch requests to OpenRouter with up to 20 questions per call. This reduces cold starts and API round-trips.
7. Use Supabase Storage for large PDFs. Instead of returning PDF data in the response body, upload to Storage and return a signed URL. This avoids response size limits and enables caching.
8. Implement connection pooling awareness. Edge Functions share the database connection pool. During high-traffic periods (exam submissions), functions should use short-lived connections and avoid long transactions.

6.3 Monitoring & Reliability

9. Set up Supabase log monitoring. Use the Supabase Dashboard logs (or the get_logs MCP tool) to monitor Edge Function errors. Set up alerts for error rate spikes.
10. Add retry logic for external API calls. OpenRouter and Resend calls should retry 3 times with exponential backoff. Network failures are common and transient.
11. Implement circuit breaker for AI functions. If OpenRouter returns errors for 5 consecutive calls, disable AI features gracefully and fall back to rule-based alternatives. Show "AI features temporarily unavailable" to users.
12. Track function execution time. Log execution duration for each function invocation. Functions consistently approaching the 150-second limit need optimization or chunking.

6.4 Security

13. Enable JWT verification on all user-facing functions. Only disable JWT for cron-triggered functions and the health check. The current inventory shows all user-facing functions correctly require JWT.
14. Validate institution_id in every function. Even with RLS, Edge Functions using the service role key bypass RLS. Every function must explicitly check that the requesting user belongs to the correct institution.
15. Rate limit AI functions per user. Implement per-user rate limiting (50 requests/hour for AI functions) to prevent abuse and cost overruns. The existing _shared/rateLimiter.ts utility supports this.
16. Never log request bodies containing student PII. Edge Function logs are accessible to project admins. Ensure no student names, emails, or assessment data appear in logs.

6.5 Cost Management

17. Monitor OpenRouter API spend weekly. Set up a spending alert at $50/month during pilot. AI costs can spike unexpectedly during exam periods when quiz generation and feedback drafts increase.
18. Use cheaper models for non-critical AI tasks. At-risk predictions and module suggestions can use GPT-4o-mini or DeepSeek instead of GPT-4o, reducing costs by 80-90% with acceptable quality.
19. Cache verified explanations aggressively. The verified_explanations table already exists. Ensure the select-adaptive-question function checks this cache before calling the LLM for explanations.
20. Track per-function invocation counts. Use Supabase logs to identify which functions are called most frequently. Optimize the top 5 by invocation count first.

6.6 Architecture Improvements

21. Consider moving cron jobs to Vercel Cron. Vercel supports cron jobs natively (vercel.json). This avoids the pg_cron + pg_net dependency and works on any Supabase plan. The api/cron/ directory already has Vercel-compatible cron handlers.
22. Add a function deployment CI/CD step. Add a GitHub Actions workflow step that deploys Edge Functions on merge to main. Currently deployment is manual via Supabase CLI or MCP.
23. Create a shared error handler. All 35 functions duplicate CORS headers and error response formatting. Extract a shared utility in _shared/ for consistent error responses.
24. Implement function versioning. When updating a function, deploy the new version alongside the old one (blue-green). This prevents downtime during deployment.

---

7. Deployment Checklist

Before deploying each batch, verify:

- [ ] All required secrets are configured in Supabase Dashboard
- [ ] Function has been tested locally with `supabase functions serve`
- [ ] JWT verification is enabled for user-facing functions
- [ ] Error handling returns structured JSON with appropriate HTTP status codes
- [ ] CORS headers are set for browser requests
- [ ] Rate limiting is implemented for AI and write-heavy functions
- [ ] Supabase advisors show no security warnings after deployment

---

8. Deployment Command Reference

Deploy a single function:

```
npx supabase functions deploy <function-name> --project-ref cdlgtbvxlxjpcddjazzx
```

Deploy all functions:

```
npx supabase functions deploy --project-ref cdlgtbvxlxjpcddjazzx
```

List deployed functions:

```
npx supabase functions list --project-ref cdlgtbvxlxjpcddjazzx
```

View function logs:

```
npx supabase functions logs <function-name> --project-ref cdlgtbvxlxjpcddjazzx
```

Edeviser Engineering -- April 2026